

COLLEGE KNOWLEDGE ASSISTANT

RAG-Based AI Document & Knowledge Assistant

Project Level: Medium | **Domain:** Artificial Intelligence / Generative AI

1. Project Overview

The **College Knowledge Assistant** is an AI-powered question-answering system that helps students, faculty, and staff find information from college documents. Users can upload documents such as college handbooks, academic regulations, course information, examination rules, hostel guidelines, placement documents, and internship instructions. The system uses **Retrieval-Augmented Generation (RAG)** to retrieve relevant information from these documents before generating an answer.

2. Objective

Build a reliable AI assistant that can answer college-related questions using the institution's own documents instead of depending only on the language model's general knowledge.

- Allow users to upload PDF and other supported documents.
- Extract and clean text from documents.
- Split long documents into meaningful chunks.
- Convert chunks into embeddings and store them in a vector database.
- Retrieve the most relevant content for each user question.
- Generate clear, context-based answers using an LLM.
- Display document/source references so answers can be verified.
- Respond safely when the requested information is not available.

3. Main Features

Feature	Description
Document Upload	Upload college PDFs, notices, regulations, handbooks, and guides.
Text Extraction	Extract readable text from uploaded documents.
Text Chunking	Break large documents into smaller searchable sections.
Embeddings	Represent document chunks as numerical vectors.
Vector Search	Find chunks semantically related to the user's question.
Context Retrieval	Send the most relevant retrieved content to the LLM.
AI Answer Generation	Generate a concise answer based on retrieved context.
Source References	Show the document/page or source used for the answer.
Unsupported Question Handling	Avoid hallucinated answers when information is missing.

4. How the RAG System Works

- Step 1 — Upload:** The user uploads college documents.
- Step 2 — Extract:** The application extracts text and removes unnecessary formatting.
- Step 3 — Chunk:** Text is divided into smaller overlapping chunks.
- Step 4 — Embed:** An embedding model converts each chunk into a vector.
- Step 5 — Store:** Vectors and metadata are stored in FAISS or Chroma.
- Step 6 — Ask:** The user enters a college-related question.
- Step 7 — Retrieve:** The system searches for the most relevant chunks.
- Step 8 — Generate:** The LLM uses retrieved context to create the answer.
- Step 9 — Cite:** The assistant displays the relevant document/source.

5. Suggested Technology Stack

Layer	Suggested Technology
Programming	Python
Backend API	FastAPI
RAG Framework	LangChain or LlamaIndex
Embeddings	Sentence Transformers or another embedding model
Vector Database	FAISS or Chroma
LLM	Gemini, OpenAI, or Anthropic
Frontend	Streamlit, React, or Next.js
Document Processing	PyPDF / PDF extraction libraries
Version Control	Git / GitHub

6. Example College Documents

- College handbook and student regulations
- Academic calendar and examination rules
- Department/course information
- Hostel rules and facilities
- Attendance and leave policies
- Placement and internship guidelines
- Scholarship and fee information
- Laboratory and library rules
- Student services and campus facilities
- Official notices and circulars

7. Example Questions

- What is the minimum attendance requirement?
- What are the rules for applying for leave?
- When are the semester examinations conducted?
- What documents are required for an internship?
- What are the hostel timings?
- What is the procedure for getting a bonafide certificate?
- What are the eligibility requirements for placement?
- Which documents are required for scholarship application?

8. Unsupported Question Handling

If the required information cannot be found in the uploaded college documents, the assistant should not invent an answer. It should clearly state that the information was not found and, when appropriate, suggest contacting the relevant college office or uploading the correct document.

9. System Architecture

User → Web Interface → Backend → Document Processor → Chunking → Embedding Model → Vector Database → Retriever → LLM → Answer + Sources

The document-processing pipeline runs when files are added or updated. The question-answering pipeline runs whenever the user asks a question. Separating these pipelines makes the application easier to maintain and scale.

10. Requirements

- Functional RAG pipeline
- Accurate retrieval from college documents
- Appropriate handling of unsupported questions
- Secure storage of API keys using environment variables
- Input validation and error handling
- Clean modular project architecture
- Source/reference display
- Professional README with installation and usage instructions

11. Expected Output

A student asks a question in the chat interface. The system searches the college knowledge base, retrieves the most relevant document sections, and returns a natural-language answer with a source reference. This reduces the time students spend manually searching through lengthy PDFs.

12. Future Enhancements

- Voice-based questions and answers
- Multi-language support
- OCR for scanned documents
- Admin dashboard for document management
- Role-based access for students, faculty, and administrators
- Conversation history
- Automatic document update detection
- Analytics for frequently asked questions

13. Conclusion

The College Knowledge Assistant demonstrates how RAG can connect generative AI with an institution's private knowledge base. By retrieving relevant college information before generating an answer, the system can provide useful, traceable, and document-grounded responses. The project is suitable as a medium-level AI/GenAI internship or academic project.